1. INTRODUCTION

This Test Plan outlines the **strategy, scope, resources, schedule,** and **activities** required to thoroughly test the Payroll Management Application. The payroll system provides end-to-end functionality for employee master data management, time and attendance processing, payroll calculations, statutory deductions, payslip generation, bank advice creation, compliance reporting, and integration with finance systems. This plan ensures the testing process is structured, repeatable, and aligned with business and compliance requirements.

2. TEST OBJECTIVES

- Validate the accuracy of gross-to-net salary calculations for all employee types.
- Verify correctness of earnings such as basic pay, allowances, overtime, and variable components.
- Validate statutory and non-statutory deductions such as income tax, PF, ESI, loans, and insurance.
- Ensure compliance rules and statutory limits are applied correctly.
- Validate integrations with HRIS, Time & Attendance, Banking, and Finance/GL systems.
- Ensure data security, role-based access, audit logs, and PII protection are functioning correctly.
- Confirm system performance under peak load during payroll periods.
- Validate usability, accessibility, and localization for currency, date formats, etc.
- Ensure disaster recovery, backup, and restore procedures work for payroll-critical data.

3. TESTING SCOPE

3.1 In-Scope

- Employee master data: create, update, terminate, rehire, transfer.
- Time & Attendance imports: regular hours, overtime, leave, holidays.
- Payroll configuration: pay groups, calendars, earning/deduction components, statutory rates.

4. TEST APPROACH / STRATEGY

A structured approach combining manual and automated testing will be used. Testing will follow risk-based prioritization and requirement traceability.

4.1 Test Types

- Unit Testing (by developers)
- API Testing (Postman/REST Assured)
- System Integration Testing (SIT)
- Functional Testing (scripted + exploratory)
- Regression Testing (manual + automated)
- Data Validation Testing
- Security Testing (RBAC, encryption, access logs)
- Performance & Load Testing (JMeter/Locust)
- User Acceptance Testing (UAT)

- Accessibility Testing
- Backup/Restore Testing

4.2 Tools

- Test Management: Azure DevOps / Jira (Xray / Zephyr)
- Defect Tracking: Azure DevOps / Jira
- UI Automation: Selenium / Playwright
- API Automation: REST-Assured / Postman
- Performance Tools: JMeter / Locust
- DB Validation: SQL-based tools
- Security Tools: OWASP ZAP

4.3 Traceability

- Requirements Traceability Matrix (RTM) will map business requirements → test cases → defects.
- Coverage metrics will be maintained.

5. TEST ENVIRONMENT

- **Hardware**: Production-simulated servers for app and database.
- **Software**: Payroll App (current build), Web Server, App Server, Database Server.

Test Plan vs Test Strategy

Aspect	Test Plan	Test Strategy
Definition	A document that outlines *what* testing will be done and *how* it will be executed for a specific project or release.	A high-level document describing the *overall approach* and *methodology* for testing across the organization or product.
Purpose	To provide detailed instructions for testers to execute testing activities for a particular project.	To define the long-term testing vision, standards, and principles fo the overall QA process.
Focus	Project-specific testing activities.	Organization-wide or product-wide testing approach.
Level of Detail	Very detailed (scope, test cases, environment, schedule, risks).	High-level (methodologies, standards, testing approach).
Scope	Narrow – applies to one project, module, or release.	Broad – applies to all projects or the entire product line.
When Created	After the requirements are finalized and before testing starts.	During the planning phase, often before the Test Plan.
Prepared By	Test Lead / QA Manager.	QA Manager / Test Architect / Senior Leadership.
Approval Authority	Project Manager / QA Manager / Business Stakeholders.	Organizational QA Leadership / Product Owners.
Document Type	Dynamic — updated frequently as the project evolves.	Static — rarely changes; updated only when major process changes occur.
Components	- Test scope (in/out) - Test environment - Test cases - Roles & responsibilities - Entry/exit criteria - Test schedule	- Testing approach - Testing methodology - Tools & frameworks - QA standards - Test levels & types